



Exercices avancés de Programmation Parallèle et Distribuée

Problème de sélection . Le problème étudié dans cet exercice est étroitement lié et appartient à une famille de problèmes appelés problèmes de comparaison. Ces problèmes sont généralement résolus en comparant des paires d'éléments d'une séquence en entrée. Pour bien introduire notre problème, nous avons besoin des définitions suivantes.

Ordre linéaire : Les éléments d'un ensemble A satisfont un ordre linéaire $<$ si et seulement si :

1. Pour tous les éléments a et b de A : $a < b$, $a = b$, ou $b < a$.
2. Pour tous les éléments a , b et c de A , si $a < b$ et $b < c$, alors $a < c$.

Le symbole $<$ est lu comme "précède". Par exemple, l'ensemble des nombres entiers ou des lettres de l'alphabet latin respectent un ordre linéaire.

Rang : Pour une séquence $S = S_1, S_2, \dots, S_N$ dont les éléments proviennent d'un ensemble ordonné linéairement, le rang d'un élément s_i est défini comme étant le nombre d'éléments qui le précèdent dans S , plus 1. Par exemple, dans $S=(8, -3, 2, -5, 6, 0)$, le rang de 0 est 3.

Sélection : Étant donné une séquence ordonnée linéairement $S = S_1, S_2, \dots, S_N$ et un entier k , où $1 \leq k \leq n$, on cherche l'élément de rang k . Par exemple, dans $S=(8, -3, 2, -5, 6, 0)$, l'élément de rang 4 est 2. Nous noterons cet élément $s_{(k)}$. Dans notre discussion, nous supposons que S est une séquence d'entiers (sans perte de généralité). Le problème de sélection revient alors à trouver le k -ième plus petit élément.

Nous introduisons également la notation suivante :

- $\lfloor r \rfloor$: plus grand entier inférieur ou égal à r (*plancher* de r).
- $\lceil r \rceil$: plus petit entier supérieur ou égal à r (*plafond* de r).

Exemples : $\lfloor 3.9 \rfloor = 3, \lceil 3.1 \rceil = 4, \lfloor 3.0 \rfloor = \lceil 3.0 \rceil = 3$.

Complexité : Trois cas particuliers du problème de sélection sont évidents :

- $k=1$: recherche du plus petit élément.
- $k=n$: recherche du plus grand élément.



Exercices avancés de Programmation Parallèle et Distribuée

- $k = \lfloor n/2 \rfloor$: recherche de la médiane, où la moitié des éléments sont plus petits et l'autre moitié plus grands.

Il est intuitivement plus facile de trouver le plus petit ou le plus grand élément en parcourant simplement la liste et en gardant en mémoire la valeur minimale ou maximale. Cependant, pour $1 < k < n$, la solution n'est pas immédiate.

Si la séquence S était déjà triée, alors la sélection serait triviale : nous pourrions obtenir $s_{(k)}$ en une seule étape. Mais trier S entièrement avant de sélectionner k serait trop coûteux en calcul.

Quel que soit k , une chose est certaine : il faut examiner chaque élément de S au moins une fois. Cela établit une borne inférieure de $\Omega(n)$ sur le nombre d'étapes nécessaires à un algorithme séquentiel. Cela implique une borne inférieure de $\Omega(n)$ sur le coût de tout algorithme parallèle de sélection.

UN ALGORITHME SÉQUENTIEL

Nous étudions maintenant un algorithme séquentiel pour le problème de sélection.

Pourquoi s'intéresser à un algorithme séquentiel ?

1. Notre algorithme parallèle sera basé sur lui.
2. L'algorithme parallèle l'utilisera comme sous-procédure.

L'algorithme, appelé SEQUENTIAL SELECT, est récursif et utilise la stratégie diviser pour régner.

Algorithme SEQUENTIAL SELECT

Entrée : une séquence S et un entier k .

Sortie : le k -ième plus petit élément de S .

1. Si $|S| < Q$, trier S et retourner directement le k -ième élément.
Sinon :
Diviser S en $|S|/Q$ sous-séquences de Q éléments chacune (Q est une constante à déterminer).
2. Trier chaque sous-séquence et calculer sa médiane.



Exercices avancés de Programmation Parallèle et Distribuée

3. Trouver la médiane mmm des médianes en appelant SEQUENTIAL SELECT récursivement.
4. Séparer S en trois sous-séquences :
 - S_1 : éléments $< m$.
 - S_2 : éléments $= m$.
 - S_3 : éléments $> m$.
5. Si $|S_1| \geq k$, rechercher récursivement dans S_1 .
Sinon, si $|S_1| + |S_2| \geq k$, retourner m.
Sinon, rechercher récursivement dans S_3 pour l'élément $(k - |S_1| - |S_2|)$ -ième.

Analyse

L'algorithme a une complexité de $O(n)$, ce qui est optimal.

Q doit être choisi pour minimiser le temps d'exécution.

Stratégies de Parallélisation

Diffusion d'une donnée

Soit D une position mémoire contenant une donnée que les N processeurs doivent recevoir à un moment donné lors de l'exécution d'un algorithme. Nous supposons la présence d'un tableau A de longueur N en mémoire. Ce tableau est initialement vide et sert d'espace de travail pour distribuer le contenu de D aux processeurs. La position i de ce tableau est notée A(i).



Exercices avancés de Programmation
Parallèle et Distribuée

Procédure BROADCAST (D, N, A)

Étape 1 : Processeur P_1

1. Lit la valeur stockée dans **D**.
2. **Stocke** cette valeur dans sa mémoire locale.
3. **Écrit** cette valeur dans **A(1)**.

Étape 2 : Diffusion en parallèle

- Pour $i = 0$ à $\log N - 1$, faire :
 - Pour $j = 2^i + 1$ à 2^{i+1} , faire en parallèle :
 - Le processeur P_j :
 1. Lit la valeur dans **A(j - 2ⁱ)**.
 2. **Stocke** cette valeur dans sa propre mémoire locale.
 3. **Écrit** cette valeur dans **A(j)**.